

MA4151 Kriptografi

Pritta Etriana Putri

Program Studi Matematika
Fakultas Matematika dan Ilmu Pengetahuan Alam
Institut Teknologi Bandung

2020

Invers Perkalian dengan Algoritma Euclid

$\mathbb{Z}_n^*$ menyatakan himpunan residu modulo n yang relatif prima terhadap n .

$\mathbb{Z}_n^*$ membentuk grup abel terhadap perkalian.

Diberikan bilangan bulat r_0 dan r_1 , dan $r_0 > r_1$. Algoritma Euclid menghitung pembagi bersama terbesar dari r_0 dan r_1 .

Algoritma Euclid

$$r_0 = q_1 r_1 + r_2, \quad 0 < r_2 < r_1$$

$$r_1 = q_2 r_2 + r_3, \quad 0 < r_3 < r_2$$

$$\vdots$$

$$r_{m-2} = q_{m-1} r_{m-1} + r_m, \quad 0 < r_m < r_{m-1}$$

$$r_{m-1} = q_m r_m.$$

Maka, mudah ditunjukkan bahwa

$$\gcd(r_0, r_1) = \gcd(r_1, r_2) = \cdots = \gcd(r_{m-1}, r_m) = r_m.$$

Sehingga, ini memberikan bahwa $\gcd(r_0, r_1) = r_m$.

Karena Algoritma Euclid menghitung $\gcd$, maka ia dapat digunakan untuk **menentukan** apakah suatu bilangan positif $b < n$ mempunyai invers perkalian modulo n , dengan mulai memilih $r_0 = n$ dan $r_1 = b$.

Akan tetapi, **algoritma ini tidak menghitung** nilai dari invers perkalian dari b (jika ada).

Sekarang, definisikan barisan bilangan $k_0, k_1, \dots, k_m$ menurut aturan rekursif berikut: (dimana q_i' s terdefinisi di atas):

$$k_0 = 0$$

$$k_1 = 1$$

$$k_i = k_{i-2} - q_{i-1}' k_{i-1} \pmod{r_0}, \text{ if } i \geq 2.$$

Maka, kita peroleh hasil penting di bawah ini.

Theorem 1.

Untuk $0 \leq i \leq m$, berlaku $r_i \equiv k_i r_1 \pmod{r_0}$, dimana q_i' s dan r_j' s didefinisikan seperti pada Algoritma Euclid, dan k_i' s didefinisikan seperti pada rekursif di atas.

Bukti. Kita buktikan dengan induksi pada i . Pernyataan benar untuk $i = 0$ dan $i = 1$. Misal pernyataan benar untuk $i = j - 1$ dan $j - 2$, dimana $j \geq 2$; kita akan menunjukkan bahwa pernyataan juga benar untuk $i = j$. Dengan induksi, kita mempunyai :

$$r_{j-2} \equiv k_{j-2}r_1 \pmod{r_0}$$

dan

$$r_{j-1} \equiv k_{j-1}r_1 \pmod{r_0}.$$

Sekarang, hitung:

$$\begin{aligned} r_j &= r_{j-2} - q_{j-1}r_{j-1} \\ &\equiv k_{j-2}r_1 - q_{j-1}k_{j-1}r_1 \pmod{r_0} \\ &\equiv (k_{j-2} - q_{j-1}k_{j-1})r_1 \pmod{r_0} \\ &\equiv k_jr_1 \pmod{r_0}. \end{aligned}$$

Sehingga, pernyataan benar dengan induksi. □

Pernyataan berikut adalah konsekuensi langsung.

Corollary 1.

Misal $\gcd(r_0, r_1) = 1$. Maka $k_m r_1 \equiv 1 \pmod{r_0}$.

Contoh 1. Misal kita ingin menghitung $28^{-1} \pmod{75}$.

Kita mengikuti proses di bawah ini:

$75 = 2 \cdot 28 + 19$	$k_0 = 0$
$28 = 1 \cdot 19 + 9$	$k_1 = 1$
$19 = 2 \cdot 9 + 1$	$k_2 = 0 - 2 \cdot 1 = -2 \equiv 73 \pmod{75}$
$9 = 9 \cdot 1$	$k_3 = 1 - 1 \cdot 73 = -72 \equiv 3 \pmod{75}$
	$k_4 = 73 - 2 \cdot 3 = 67 \pmod{75}$

Sehingga, $28^{-1} \pmod{75} = 67$.



Theorema "Chinese Remainder"

Misal $m_1, m_2, \dots, m_r$ bilangan bulat yang setiap pasanganya relatif prima. Yakni, $\gcd(m_i, m_j) = 1$, jika $i \neq j$. Misal $a_1, \dots, a_r$ bilangan bulat, dan pandang sistem kongruensi berikut ini:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$\vdots$$

$$x \equiv a_r \pmod{m_r}.$$

Theorema "Chinese Remainder" menunjukkan bahwa sistem di atas mempunyai **solusi tunggal** modulo $M = m_1 m_2 \cdots m_r$.

Perhatikan fungsi $\alpha : \mathbb{Z}_M \rightarrow \mathbb{Z}_{M_1} \times \cdots \times \mathbb{Z}_{M_r}$, yang didefinisikan sbb:

$$\alpha(x) = (x \bmod m_1, \cdots, x \bmod m_r).$$

Contoh 2. Misal $r = 2$, $m_1 = 4$ dan $m_2 = 3$, sehingga $M = 12$. Maka, fungsi α mempunyai nilai berikut:

$$\begin{array}{lll} \alpha(0) = (0, 0) & \alpha(1) = (1, 1) & \alpha(2) = (2, 2) \\ \alpha(3) = (3, 0) & \alpha(4) = (0, 1) & \alpha(5) = (1, 2) \\ \alpha(6) = (2, 0) & \alpha(7) = (3, 1) & \alpha(8) = (0, 2) \\ \alpha(9) = (1, 0) & \alpha(10) = (2, 1) & \alpha(11) = (3, 2). \quad \square \end{array}$$

Dalam menunjukkan kebenaran Teorema "Chinese Remainder", pertama buktikan bahwa fungsi α satu-satu dan pada. Pada Contoh 2, hal ini dapat dilihat dengan mudah.

Secara umum, kita akan memberikan formula eksplisit dari fungsi inversnya, yakni α^{-1} .

Untuk $1 \leq j \leq r$, definisikan

$$M_j = \frac{M}{m_j}$$

Maka, mudah dilihat bahwa

$$\gcd(M_j, m_j) = 1, \text{ untuk } 1 \leq j \leq r.$$

Selanjutnya, untuk $1 \leq j \leq r$, misal

$$y_j = M_j^{-1} \pmod{m_j}.$$

Ini berarti bahwa

$$M_j y_j \equiv 1 \pmod{m_j}, \text{ untuk } 1 \leq j \leq r.$$

Sekarang, definisikan fungsi $\gamma : \mathbb{Z}_{M_1} \times \mathbb{Z}_{M_2} \times \cdots \times \mathbb{Z}_{M_r} \rightarrow \mathbb{Z}_M$ sbb:

$$\gamma(a_1, a_2, \dots, a_r) = \sum_{i=1}^r a_i M_i y_i \pmod{M}.$$

Kita akan tunjukkan bahwa fungsi $\gamma = \alpha^{-1}$. Ini memberikan formula eksplisit untuk memecahkan sistem kongruensi di atas.

Nyatakan $X = \gamma(a_1, a_2, \dots, a_r)$ dan misalkan $1 \leq j \leq r$. Pandang bentuk $a_i M_i y_i$ dalam modulo m_j : Jika $i = j$ maka

$$a_i M_i y_i \equiv a_i \pmod{m_j}.$$

karena

$$M_i y_i \equiv 1 \pmod{m_j}.$$

Di sisi lain, jika $i \neq j$, maka

$$a_i M_i y_i \equiv 0 \pmod{m_j}.$$

sejak $m_j | M_i$ dalam hal ini. Sehingga, didapat

$$\begin{aligned} X &\equiv \sum_{i=1}^r a_i M_i y_i \pmod{m_j} \\ &\equiv a_j \pmod{m_j}. \end{aligned}$$

Karena ini berlaku untuk semua j , $1 \leq j \leq r$, X adalah solusi dari sistem kongruensi tersebut.

Selanjutnya, kita perlu menunjukkan bahwa solusi X tunggal modulo M . Ini dapat dilakukan dengan penghitungan sederhana. Fungsi α adalah suatu fungsi yang mempunyai domain berkardinalitas M dan mempunyai daerah hasil berukuran M . Kita telah menunjukkan bahwa α pada (onto). Sehingga, α haruslah satu-satu, karena domain dan daerah hasil berukuran sama. Ini memberikan bahwa α bijektif (satu-satu dan pada) dan $\alpha^{-1} = \gamma$.

Contoh 3. Misal $r = 3$, $m_1 = 5$, $m_2 = 11$ dan $m_3 = 13$. Maka, $M = 715$. Kita hitung $M_1 = 143$, $M_2 = 65$ dan $M_3 = 55$, dan maka $y_1 = 2$, $y_2 = 10$ dan $y_3 = 9$. Maka, fungsi $\alpha^{-1} : \mathbb{Z}_5 \times \mathbb{Z}_{11} \times \mathbb{Z}_{13} \rightarrow \mathbb{Z}_{715}$ terdefinisi sbb :

$$\alpha^{-1}(a_1, a_2, a_3) = 286a_1 + 650a_2 + 495a_3 \pmod{715}.$$

Sebagai contoh, jika $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{11}$, $x \equiv 7 \pmod{13}$, maka formula di atas didapat

$$\begin{aligned} x &= 286 \times 3 + 650 \times 2 + 495 \times 7 \pmod{715} \\ &= 5623 \pmod{715} \\ &= 618 \pmod{715}. \quad \square \end{aligned}$$

Teorema "Chinese Remainder"

Misal $m_1, \dots, m_r$ bilangan bulat yang setiap pasanganya relatif prima, dan misal $a_1, \dots, a_r$ bilangan bulat. Maka, sistem kongruensi:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$\vdots$$

$$x \equiv a_r \pmod{m_r}.$$

punya SOLUSI TUNGGAL modulo $M = m_1 \times m_2 \times \dots \times m_r$, yang diberikan oleh:

$$x = \sum_{i=1}^r a_i M_i y_i \pmod{M},$$

dimana $M_i = M/m_i$ dan $y_i = M_i^{-1} \pmod{m_i}$, untuk $i \leq r$.

Teorema Lagrange

Untuk suatu grup perkalian (hingga) G , definisikan **order** dari suatu elemen $g \in G$ sebagai bilangan positif terkecil k sehingga $g^k = 1$.

Theorem 3. (*Lagrange*)

Misal G suatu grup perkalian berukuran n , dan $g \in G$. Maka, order dari elemen g senantiasa MEMBAGI n .

Corollary 2.

Jika $b \in \mathbb{Z}_n^*$, maka $b^{\phi(n)} \equiv 1 \pmod{n}$.

Bukti. $\mathbb{Z}_n^*$ adalah grup perkalian berukuran $\phi(n)$. □

Corollary 3. (Fermat)

Misal p bilangan prima dan $b \in \mathbb{Z}_p$. Maka, $b^p \equiv b \pmod{p}$.

Bukti. Jika p prima, maka $\phi(p) = p - 1$. Maka, untuk $b \not\equiv 0 \pmod{p}$, pernyataan benar dari Corollary 2. Untuk $b \equiv 0 \pmod{p}$, pernyataan juga benar karena $0^{p-1} \equiv 0 \pmod{p}$. □

Pada titik ini, kita tahu bahwa jika p prima maka $\mathbb{Z}_p^*$ merupakan grup berorder $p - 1$, dan setiap elemen di $\mathbb{Z}_p^*$ punya order yang membagi $p - 1$. Lebih lanjut, jika p prima maka grup $\mathbb{Z}_p^*$ **siklis**; yakni, terdapat suatu elemen $\alpha \in \mathbb{Z}_p^*$ yang berorder persis sama dengan $p - 1$. Sehingga, kita memiliki:

Theorem 4.

Jika p prima maka $\mathbb{Z}_p^*$ grup siklis.

Suatu elemen α yang berorder $p - 1$ disebut elemen primitif modulo p .
Amati bahwa α primitif jika dan hanya jika

$$\{\alpha^i : 0 \leq i \leq p - 2\} = \mathbb{Z}_p^*.$$

Misal p prima, α primitif modulo p . Setiap elemen $\beta \in \mathbb{Z}_p^*$ dapat dituliskan sbb:

$$\beta = \alpha^i,$$

dimana $0 \leq i \leq p - 2$, dalam suatu cara yg TUNGGAL.

Adalah tidak sukar utk membuktikan bahwa order dari $\beta = \alpha^i$ adalah

$$\frac{p-1}{\gcd(p-1, i)}.$$

Maka, β sendiri adalah elemen primitif jika dan hanya jika $\gcd(p-1, i) = 1$.

Hal ini memberikan konsekuensi bahwa banyaknya elemen primitif modulo p adalah $\phi(p-1)$.

Contoh 4. Misal $p = 13$. Dengan menentukan pangkat dari 2 secara berturut-turut, kita dapat memeriksa bahwa 2 elemen primitif modulo 13:

$$2^0 \pmod{13} = 1$$

$$2^1 \pmod{13} = 2$$

$$2^2 \pmod{13} = 4$$

$$2^3 \pmod{13} = 8$$

$$2^4 \pmod{13} = 3$$

$$2^5 \pmod{13} = 6$$

$$2^6 \pmod{13} = 12$$

$$2^7 \pmod{13} = 11$$

$$2^8 \pmod{13} = 9$$

$$2^9 \pmod{13} = 5$$

$$2^{10} \pmod{13} = 10$$

$$2^{11} \pmod{13} = 7.$$

Elemen 2^i primitif jika dan hanya jika $\gcd(i, 12) = 1$; yakni jika dan hanya jika $i = 1, 5, 7$ atau 11. Sehingga, elemen-elemen primitif modulo 13 adalah 2, 6, 7, dan 11.

Sistem Kripto RSA

Sistem kripto RSA dapat digambarkan pada tabel di bawah. Sistem kripto ini menggunakan komputasi dalam $\mathbb{Z}_n$, dimana $n = pq$, p, q prima. Catat bahwa $\phi(n) = (p - 1)(q - 1)$.

Misal $n = pq$, p dan q prima. Misal $\mathcal{P} = \mathcal{C} = \mathbb{Z}_n$, definisikan

$$K = \{(n, p, q, a, b) : n = pq, p, q \text{ prima}, ab \equiv 1 \pmod{\phi(n)}\}.$$

Untuk $K = \{(n, p, q, a, b)\}$, definisikan

$$e_K(x) = x^b \pmod{n}$$

dan

$$d_K(y) = y^a \pmod{n}$$

dimana $x, y \in \mathbb{Z}_n$. Nilai n dan b kunci **publik** dan nilai p, q, a **rahasia**.

Periksa bahwa enkripsi dan dekripsi merupakan operasi yang saling invers. Karena

$$ab \equiv 1 \pmod{\phi(n)},$$

didapat $ab = t\phi(n) + 1$ untuk suatu bilangan bulat $t \geq 1$. Untuk **setiap** $x \in \mathbb{Z}_n$, diperoleh bahwa:

$$\begin{aligned}(x^b)^a &\equiv x^{t\phi(n)+1} \pmod{p}, \\ &\equiv x^{t(p-1)(q-1)+1} \pmod{p}, \\ &\equiv (x^{p-1})^{t(q-1)} x \pmod{p}, \\ &\equiv 1^{t(q-1)} x \pmod{p}, \text{ (Teorema Fermat)}, \\ &\equiv x \pmod{p}.\end{aligned}$$

Dengan cara serupa, didapat

$$(x^b)^a \equiv x \pmod{q}$$

Maka, dengan Teorema "Chinese Remainder", dapat disimpulkan:

$$(x^b)^a \equiv x \pmod{pq}$$

Contoh 5. Misal Bob memilih $p = 101$, dan $q = 113$. Maka $n = 11413$ dan $\phi(n) = 100 \times 112 = 11200$. Karena $11200 = 2^6 5^2 7$, bilangan b dapat digunakan sebagai eksponen enkripsi jika dan hanya jika b tidak terbagi oleh 2, 5 atau 7. (Dalam praktek, Algoritma Euclid digunakan untuk mencari b yang memenuhi $\gcd(\phi(n), b) = 1$).

Misal Bob memilih $b = 3533$. Maka, didapat $a = b^{-1} = 6597 \pmod{11200}$. Bob mempublikasikan $n = 11413$ dan $b = 3533$ dalam direktorinya.

Sekarang misal Alice ingin mengirim plaintext 9726 ke Bob. Ia menghitung

$$9726^{3533} \pmod{11413} = 5761,$$

dan mengirim katasandi 5761 pada kanal. Ketika Bob menerima katasandi 5761, ia menghitung

$$5761^{6597} \pmod{11413} = 9726. \quad \square$$

Keamanan dari **RSA** didasarkan pada harapan bahwa fungsi enkripsi $e_K(x) = x^b \pmod{n}$ *one-way*, sehingga ini *computationally infeasible* untuk semua lawan yang berusaha men-decrypt katasandi. Pengetahuan tentang faktorisasi $n = pq$ adalah *trapdoor* yang dapat dipakai oleh Bob untuk meng-decrypt. Dengan faktorisasi ini, ia dapat menghitung $\phi(n) = (p-1)(q-1)$, dan maka menghitung eksponen dekripsi a .

Implementasi RSA

Untuk men-set up sistem RSA, dapat dilakukan tahap - tahap berikut:

- 1 Pilih dua bilangan prima besar, p dan q .
- 2 Hitung $n = pq$ dan $\phi(n) = (p - 1)(q - 1)$.
- 3 Pilih bil. random $b(1 < b < \phi(n))$, dan $\gcd(b, \phi(n)) = 1$.
- 4 Hitung $a = b^{-1} \bmod \phi(n)$.
- 5 *Publish* n dan b sebagai kunci publik.

Salah satu serangan yang mungkin pada **RSA** adalah usaha untuk melakukan faktorisasi n . Jika hal ini dapat dilakukan maka orang dapat menghitung $\phi(n) = (p - 1)(q - 1)$; dan kemudian menghitung a dari b .

Dengan demikian, usaha memecahkan sandi **RSA** ekuivalen secara polinomial dengan usaha memfaktorkan n . Akan tetapi, ekuivalensi ini masih belum dapat dibuktikan. Agar **RSA** aman, maka bilangan $n = pq$ harus dipilih cukup besar sehingga faktorisasi n akan *computationally infeasible*.

Algoritma-algoritma faktorisasi saat ini dapat memfaktorkan bilangan dengan maksimal 130 desimal digit. Sehingga, agar aman kita harus memilih p dan q masing - masing bilangan dengan 100 digit; Maka n akan punya 200 digit.

Beberapa implementasi hardware dari **RSA** menggunakan bilangan n dengan 512-bit (dengan 154-desimal digit). Karena n sangat besar, maka kita harus menggunakan aritmatika multipresisi utk melakukan komputasi dalam $\mathbb{Z}_n$, dan waktu yg dibutuhkan akan tergantung pada jumlah bit dalam representasi biner dari n .

Misal n punya k bit dalam representasi binernya, yakni $k = \lfloor \log_2 n \rfloor + 1$. Maka,

Operasi pada dua bil. bulat k -bit	Waktu
Penjumlahan	$\mathcal{O}(k)$
Perkalian	$\mathcal{O}(k^2)$
Reduksi modulo n	$\mathcal{O}(k^2)$
Perkalian modular: $xy \pmod{n}$	$\mathcal{Z}(k^2)$

Sekarang, perhatikan **perpangkatan modular (modular exponentiation)**, yakni, komputasi dari fungsi $x^b \pmod{n}$. Enkripsi dan dekripsi menggunakan fungsi tersebut. Komputasi $x^b \pmod{n}$ dapat dilakukan dgn menggunakan $b - 1$ buah perkalian modular; Akan tetapi hal ini **sangat tidak efisien** jika b besar.

Metode sangat terkenal **kuadrat-dan-kali** ("square-and-multiply") di bawah ini dapat digunakan utk menghitung $x^b \pmod n$ dalam paling banyak $2l$, l menyatakan banyaknya bit yg dibutuhkan oleh b . Sejak $l \leq k$, maka operasi $x^b \pmod n$ dapat dikomputasi dalam waktu $O(k^3)$.

Metode kuadrat-dan-kali utk menghitung $x^b \pmod n$

0. Misal $\sum_{i=0}^{l-1} b_i 2^i$.

1. $z = 1$.

2. **for** $i = l - 1$ **downto** 0 **do**.

3. $z = z^2 \pmod n$.

4. **if** $b_i = 1$ **then** $z = z \times x \pmod n$.

Pengujian Primalitas Probabilistik

Dalam setting RSA kita perlu untuk men-generate bilangan "prima random" yang besar. Dalam praktek, cara yang dilakukan adalah men-generate bilangan random besar, dan kemudian mengecek bilangan tersebut apakah prima atau tidak dengan menggunakan algoritma probabilistik yang *polynomial-time*.

Dalam bagian ini, kita mengkaji algoritma Monte Carlo seperti Algoritma Solovay-Strassen dan Miller-Rabin.

Pertanyaan: Berapa banyak bilangan random yg perlu diperiksa sampai kita mendapatkan satu yang prima?

Banyaknya bilangan prima yg tidak melebihi N adalah

$$\frac{N}{\ln N}.$$

Sehingga, jika p dipilih random, probabilitas bahwa p prima adalah sebesar $1/\ln p$. Untuk bilangan prima 512-bit, didapat $1/\ln p \approx 1/177$. Ini berarti bahwa, secara rata-rata, dari 177 buah bilangan random p dgn ukuran yg sesuai, salah satu akan prima (Jika kita hanya memperhatikan bilangan yg ganjil saja, maka probabilitas akan meningkat (double) menjadi kira-kira $2/177$).

A **Problem keputusan (decision problem)** adalah suatu problem yang meminta jawaban "ya" atau "tidak" dalam penyelesaiannya. **Algoritma probabilistik** adalah suatu algoritma yang menggunakan bilangan random. Sedangkan, algoritma yang tidak menggunakan bilangan random disebut **algoritma deterministik**.

Definisi 1.

Algoritma Monte Carlo "yes-biased" adalah algoritma probabilistik untuk problem keputusan dimana jawaban "yes" yang diberikan (selalu) benar, akan tetapi jawaban "no" yang diberikan mungkin tidak benar. Algoritma Monte Carlo "no-biased" didefinisikan serupa. Kita dapat mengatakan bahwa algoritma Monte Carlo yes-biased punya probabilitas kesalahan sama dengan δ jika untuk setiap *instance* yg mempunyai jawaban "yes", algoritma akan memberikan jawaban (tidak benar) "no" dengan probabilitas paling banyak δ .

Problem **Komposit** di bawah adalah problem keputusan.

Problem Komposit. Bilangan bulat positif $n \geq 2$.

Pertanyaan: Apakah n komposit?

Pertama, kita akan menjabarkan algoritma Solovay-Strassen, dimana merupakan algoritma Monte Carlo untuk **Problem Komposit** dengan probabilitas kesalahan $1/2$. Sehingga, jika algoritma menjawab "yes" maka n pasti komposit; Sebaliknya, jika n komposit, maka algoritma menjawab "yes" dengan probabilitas sedikitnya $1/2$. Sebelum kita membahas algoritmanya, perhatikan beberapa konsep teori bilangan berikut ini.

Definisi 2.

Misal p prima ganjil dan x bulat, $1 \leq x \leq p-1$. Bilangan x disebut **residu kuadratik modulo p** jika kongruensi $y^2 \equiv x \pmod{p}$ punya solusi $y \in \mathbb{Z}_p$. Bilangan x disebut **non-residu kuadratik modulo p** jika $x \not\equiv 0 \pmod{p}$ dan x bukan residu kuadratik modulo p .

Contoh 6. Residu kuadratik modulo 11 adalah 1, 3, 4, 5 dan 9. Catat bahwa $(\pm 1)^2 = 1$, $(\pm 5)^2 = 3$, $(\pm 2)^2 = 4$, $(\pm 4)^2 = 5$, dan $(\pm 3)^2 = 9$ (modulo 11).

Problem **Residu kuadrat** di bawah adalah problem keputusan.

Problem Instance. Bilangan prima ganjil p , dan bulat x sehingga $0 \leq x \leq p - 1$.

Pertanyaan: Apakah x residu kuadrat modulo p ?

Berdasarkan Euler's Criterion di bawah kita dapat mendisain algoritma deterministik *polynomial-time* untuk **Residu Kuadrat**.

Teorema 5.

(*Euler's Criterion*) Misal p prima ganjil. Maka, x residu kuadrat modulo p jika dan hanya jika

$$x^{(p-1)/2} \equiv 1 \pmod{p}.$$

Bukti. ($\Rightarrow$) Misal $x \equiv y^2 \pmod{p}$. Dengan Fermat, jika p prima maka $x^{p-1} \equiv 1 \pmod{p}$ untuk setiap $x \not\equiv 0 \pmod{p}$. Maka, didapat

$$\begin{aligned} x^{(p-1)/2} &\equiv (y^2)^{(p-1)/2} \pmod{p} \\ &\equiv (y)^{(p-1)} \pmod{p} \\ &\equiv 1 \pmod{p} \end{aligned}$$

($\Leftarrow$) Misal $x^{(p-1)/2} \equiv 1 \pmod{p}$. Misal b elemen primitif modulo p . Maka, $x \equiv b^i \pmod{p}$ untuk suatu i . Sehingga, diperoleh:

$$\begin{aligned} x^{(p-1)/2} &\equiv (b^i)^{(p-1)/2} \pmod{p} \\ &\equiv (y)^{i(p-1)} \pmod{p} \end{aligned}$$

Sejak b berorder $p-1$, maka pastilah $p-1$ membagi $i(p-1)/2$. Sehingga, i harus genap, dan sehingga akar kuadrat dari x adalah $\pm b^{i/2}$. $\square$

Teorema 5 menyediakan algoritma polynomial-time untuk Residu kuadrat, dgn menggunakan teknik "kuadrat-dan-kali" untuk perpangkatan *exponentiation* modulo p . Kompleksitas dari algoritma tsb sama dengan $\mathcal{O}(\log^3 p)$.

Definisi 3.

Misal p prima ganjil. Untuk sebarang bulat $a \geq 0$, definisikan **simbol Legendre** $\left(\frac{a}{p}\right)$ sbb:

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{jika } a \equiv 0 \pmod{p} \\ 1 & \text{jika } a \text{ residu kuadrat modulo } p \\ -1 & \text{jika } a \text{ non-residu kuadrat modulo } p \end{cases}.$$

Kita tahu bahwa $a^{(p-1)/2} \equiv 1 \pmod{p}$ jika dan hanya jika a residu kuadrat modulo p . Jika a kelipatan dari p maka $a^{(p-1)/2} \equiv 0 \pmod{p}$. Akhirnya, jika a non-residu kuadrat modulo p , maka $a^{(p-1)/2} \equiv -1 \pmod{p}$, karena $a^{p-1} \equiv 1 \pmod{p}$. Sehingga, kita mempunyai teorema berikut.

Teorema 6.

Misal p prima ganjil. Maka,

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}.$$



Definisi 4.

Misal n bulat ganjil dan $n = p_1^{e_1} \cdots p_k^{e_k}$. Untuk sebarang bulat $a \geq 0$, definisikan **simbol Jacobi** $\left(\frac{a}{n}\right)$ sbb:

$$\left(\frac{a}{n}\right) = \prod_{i=1}^k \left(\frac{a}{p_i}\right)^{e_i}.$$

Contoh 7. Untuk menghitung simbol Jacobi $\left(\frac{6278}{9975}\right)$, faktorkan $9975 = 3 \times 5^2 \times 7 \times 19$. Maka, diperoleh

$$\begin{aligned}\left(\frac{6278}{9975}\right) &= \left(\frac{6278}{3}\right) \left(\frac{6278}{5}\right)^2 \left(\frac{6278}{7}\right) \left(\frac{6278}{19}\right) \\ &= \left(\frac{2}{3}\right) \left(\frac{3}{5}\right)^2 \left(\frac{6}{7}\right) \left(\frac{8}{19}\right) \\ &= (-1)(-1)^2(-1)(-1) \\ &= (-1).\end{aligned}$$

Jika n prima maka $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}$, untuk **sebarang** a . Pada sisi lain, jika n komposit, maka $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}$ mungkin atau tidak mungkin berlaku. Jika persamaan ini berlaku maka n disebut **prima-semu**

Euler (Euler pseudo-prime) dengan basis a . Sebagai contoh, 91 adalah prima-semu Euler dengan basis 10, karena

$$\left(\frac{10}{91}\right) = -1 = 10^{45} \pmod{91}.$$

Untuk bilangan komposit ganjil n , n prima-semu Euler dalam basis a untuk paling banyak separuh bilangan - bilangan bulat a dimana $1 \leq a \leq n-1$. Kenyataan ini menunjukkan bahwa test primalitas Solovay-Strassen, yg diberikan pada tabel di bawah, adalah algoritma Monte Carlo "yes-biased" dengan probabilitas kesalahan maksimal $1/2$, dan dengan waktu komputasi $\mathcal{O}((\log n)^3)$.

Test primalitas Solovay-Strassen untuk bulat ganjil n

- 1 Pilih bulat random a , $1 \leq a \leq n - 1$
- 2 If $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}$ then
 jawab " n prima"
 else
 jawab " n komposit"

Dalam hal menghitung simbol Jacobi $\left(\frac{a}{n}\right)$, kita harus memfaktorkan n . Jika kita dapat memfaktorkan n , maka kita telah tahu apakah n prima atau tidak, sehingga teknik ini berakhir pada lingkaran tak berujung-pangkal (*vicious cycle*).

Untungnya, kita dapat mengevaluasi simbol Jacobi dengan tanpa pemfaktoran n , yakni dengan memakai beberapa hasil dalam teori bilangan di bawah ini:

- ① Jika n ganjil dan $m_1 \equiv m_2 \pmod{n}$ maka

$$\left(\frac{m_1}{n}\right) = \left(\frac{m_2}{n}\right).$$

- ② Jika n ganjil maka

$$\left(\frac{2}{n}\right) = \begin{cases} 1 & \text{jika } n \equiv \pm 1 \pmod{8} \\ -1 & \text{jika } n \equiv \pm 3 \pmod{8}. \end{cases}$$

- ③ Jika n ganjil maka

$$\left(\frac{m_1 m_2}{n}\right) = \left(\frac{m_1}{n}\right) \left(\frac{m_2}{n}\right).$$

- ④ Jika m dan n ganjil maka

$$\left(\frac{m}{n}\right) = \begin{cases} -\left(\frac{n}{m}\right) & \text{jika } m \equiv n \equiv 3 \pmod{4} \\ \left(\frac{n}{m}\right) & \text{lainnya.} \end{cases}$$

Contoh 8. Di bawah ini kita menghitung simbol Jacobi $\left(\frac{1872}{7411}\right)$ dengan memakai keempat sifat di atas.

$$\left(\frac{1872}{7411}\right) = \left(\frac{2}{7411}\right)^2 \left(\frac{117}{7411}\right) \quad \text{krna sifat 3}$$

$$= \left(\frac{117}{7411}\right) \quad \text{krna sifat 2}$$

$$= \left(\frac{7411}{117}\right) \quad \text{krna sifat 4}$$

$$= \left(\frac{40}{117}\right) \quad \text{krna sifat 1}$$

$$= \left(\frac{2}{117} \right)^2 \left(\frac{5}{117} \right)$$

krna sifat 3

$$= - \left(\frac{5}{117} \right)$$

krna sifat 2

$$= - \left(\frac{117}{5} \right)$$

krna sifat 4

$$= - \left(\frac{2}{5} \right)$$

krna sifat 1

$$= 1$$

krna sifat 2.

Test primalitas Miller-Rabin untuk bulat ganjil n diberikan di bawah ini. Test ini juga disebut sebagai "**test prima-semu yang kuat (strong pseudo-prime test)**", dengan probabilitas kesalahan $1/4$.

- ① Tulis $n - 1 = 2^k m$, dimana m ganjil
- ② Pilih bil. bulat random a , $1 \leq a \leq n - 1$
- ③ Hitung $b = a^m \pmod{n}$
- ④ If $b \equiv 1 \pmod{n}$ then
jawab " n prima" dan **quit**
- ⑤ For $i = 0$ then $k - 1$ do
- ⑥ If $b \equiv -1 \pmod{n}$ then
jawab " n prima" dan **quit**
else
 $b = b^2 \pmod{n}$
- ⑦ Jawab " n komposit".

Theorem 7.

Algoritma Miller-Rabin untuk komposit di atas merupakan algoritma Monte Carlo "yes-biased".

Bukti. Misal bahwa algoritma ini menjawab " n komposit" untuk suatu bilangan prima n . Maka, selanjutnya akan kita tunjukkan bahwa hal ini tidak mungkin (terdapat kontradiksi). Karena algoritma menjawab " n komposit", maka jelas bahwa $a^m \not\equiv 1 \pmod{n}$.

Perhatikan bahwa barisan nilai b yang diperiksa dalam algoritma ini. Karena b dikuadratkan pada setiap iterasi pada loop **for**, maka nilai b yang diperiksa adalah $a^m, a^{2m}, \dots, a^{2^{k-1}m}$. Sehingga dapat disimpulkan bahwa

$$a^{2^j m} \not\equiv -1 \pmod{n}$$

untuk $0 \leq j \leq k - 1$.

Selanjutnya, gunakan asumsi bahwa n prima dan dengan Teorema Fermat, kita mendapatkan

$$a^{2^k m} \equiv 1 \pmod{n}$$

Maka $a^{2^{k-1}m}$ adalah akar kuadrat (*square root*) dari 1 modulo n . Karena n prima, terdapat tepat dua akar kuadrat dari 1 modulo n , yaitu $\pm 1 \pmod{n}$. Ini dapat dilihat sbb: x akar kuadrat dari 1 modulo n jika dan hanya jika

$$n \mid (x - 1)(x + 1).$$

Sejak n prima, maka $n \mid (x - 1)$ (menyebabkan $x \equiv 1 \pmod{n}$) atau $n \mid (x + 1)$ (memberikan $x \equiv -1 \pmod{n}$).

Dari atas, kita punya bahwa

$$a^{2^{k-1}m} \not\equiv -1 \pmod{n},$$

sehingga hal ini memberikan:

$$a^{2^{k-1}m} \equiv 1 \pmod{n}.$$

Maka $a^{2^{k-2}m}$ haruslah akar kuadrat dari 1. Dengan argumentasi yang sama,

$$a^{2^{k-2}m} \equiv 1 \pmod{n}.$$

Mengulang alasan di atas, didapat

$$a^m \equiv 1 \pmod{n}.$$

suatu kontradiksi, karena algoritma akan menjawab " n prima". □

Pertanyaan: Apakah ada serangan pada RSA selain usaha faktorisasi n ?

Amati bahwa untuk melakukan serangan pada RSA, seorang kriptanalisis cukup hanya dengan menghitung $\phi(n)$. Jika n dan $\phi(n)$ diketahui, dan n merupakan hasil kali dua prima p, q , maka n dapat difaktorisasi dengan cara menyelesaikan dua persamaan:

$$\begin{aligned} n &= pq \\ \phi(n) &= (p-1)(q-1) \end{aligned}$$

untuk dua "unknown" p dan q . Substitusi $q = n/p$ ke dalam persamaan kedua, didapat

$$p^2 - (n - \phi(n) + 1)p + n = 0.$$

Dua akar dari persamaan ini adalah p dan q , faktor dari n . Sehingga, jika seorang kriptanalisis dapat mengetahui nilai $\phi(n)$, maka ia dapat memecahkan sistem. Dengan kata lain, komputasi $\phi(n)$ tidak lebih mudah dari faktorisasi n .

Eksponen Dekripsi

Kita akan menunjukkan hasil yg sangat menarik bahwa *Setiap algoritma yg menghitung eksponen dekripsi a dapat digunakan sebagai subroutine (atau oracle) dalam suatu algoritma probabilistik yang memfaktorkan n.*

Sehingga, kita dapat mengatakan bahwa "melakukan komputasi a tidak lebih mudah dari usaha faktorisasi n ". Akan tetapi, hal ini tidak menutup kemungkinan usaha untuk memecahkan (*breaking*) sistem krypto dengan tanpa menghitung a .

Definisi 5.

Misal $0 \leq \delta \leq 1$ bilangan real. Algoritma **Las Vegas** adalah suatu algoritma probabilistik dimana, untuk setiap *problem instance* I , algoritma mungkin gagal untuk memberikan jawaban dengan probabilitas δ (yakni, algoritma dapat berakhir dengan pesan "tidak ada jawaban (no answer)"). Akan tetapi, jika algoritma memberikan jawaban, maka jawaban tersebut pasti benar.

Jika kita punya algoritma Las Vegas utk memecahkan masalah, maka kita perlu me-run algoritma berkali-kali sampai mendapatkan suatu jawaban. Probabilitas bahwa algoritma kembali dengan tanpa jawaban sebanyak m kali berturut-turut adalah δ^m . Nilai rata-rata dari banyaknya kali algoritma harus dijalankan hingga mendapatkan suatu jawaban adalah $\frac{1}{1-\delta}$.

Misal **A**: suatu algoritma hipotetikal (*hypothetical algo- rithm*) yg menghitung eksponen dekripsi a dari b dan n . Kita akan memberikan suatu algoritma Las Vegas yg menggunakan **A** sebagai subrutin. Algoritma ini akan memfaktorkan n dengan probabilitas sedikitnya $1/2$. Sehingga, jika algoritma dijalankan sebanyak m kali maka n akan dapat difaktorkan dengan probabilitas sedikitnya $1 - 1/2^m$.

Algoritma didasarkan pada akar-akar kuadrat dari 1 modulo n . Misal $n = pq$. Maka, $x^2 \equiv 1 \pmod{p}$ punya dua solusi modulo p , yaitu $x = \pm 1 \pmod{p}$. Dengan cara serupa, $x^2 \equiv 1 \pmod{q}$ punya $x = \pm 1 \pmod{q}$ sebagai solusinya.

Sekarang, karena $x^2 \equiv 1 \pmod{n}$ jika dan hanya jika $x^2 \equiv 1 \pmod{p}$ dan $x^2 \equiv 1 \pmod{q}$, hal ini memberikan bahwa $x^2 \equiv 1 \pmod{n}$ jika dan hanya jika $x = \pm 1 \pmod{p}$ dan $x = \pm 1 \pmod{q}$. Sehingga, **terdapat 4 (empat) akar kuadrat dari 1 modulo n** . Dua diantaranya adalah $x = \pm 1 \pmod{n}$, disebut akar kuadrat **trivial** dari 1 modulo n . Dua akar kuadrat lainnya disebut **non-trivial**, dan mereka saling negatif satu sama lainnya modulo n .

Contoh 9. Misal $n = 403 = 13 \times 31$. Empat akar dari 1 modulo 403 adalah 1, 92, 311 dan 402. Akar kuadrat 92 diperoleh dari pemecahan sistem $x \equiv 1 \pmod{13}$, $x \equiv -1 \pmod{31}$ dengan memakai Teorema "Chinese remainder". Akar kuadrat non-trivial lainnya adalah $403 - 92 = 311$, solusi dari sistem $x \equiv -1 \pmod{13}$, $x \equiv 1 \pmod{31}$. $\square$

Misal x akar kuadrat non-trivial dari 1 modulo n . Maka, diperoleh

$$n \mid (x - 1)(x + 1),$$

tetapi n tidak membagi salah satu dari faktor di sisi kanan. Ini berakibat $\gcd(x + 1, n) = p$ atau q (dan dengan cara serupa, $\gcd(x - 1, n) = p$ atau q). Jelas, pembagi bersama terbesar dapat dihitung dengan algoritma Euclid. Sehingga, **pengetahuan akar kuadrat non-trivial dari 1 modulo n akan memberikan faktorisasi n .**

Di bawah ini kita akan menyajikan algoritma yang menggunakan algoritma hipotetikal **A** utk memfaktorkan n dgn mencari akar kuadrat non-trivial dari $1 \bmod n$.

Algoritma Pemfaktoran, diberikan eksponen dekripsi a

1. Pilih w secara random, $1 \leq w \leq n - 1$
2. Hitung $x = \gcd(w, n)$
3. **If** $1 < x < n$ **then quit** (success: $x = p$ atau q)
4. Hitung $a = A(b)$

5. Tulis $ab - 1 = 2^r s$, s ganjil
6. Hitung $v = w^s \pmod{n}$
7. **If** $v \equiv 1 \pmod{n}$ **then quit** (failure)
8. **While** $v \not\equiv 1 \pmod{n}$ **do**
9. $v_0 = v$
10. $v = v^2 \pmod{n}$
11. **If** $v_0 \equiv -1 \pmod{n}$ **then**
 quit (failure)
 else
 Hitung $x = \gcd(v_0 + 1, n)$ (success: $x = p$ atau $x = q$)

Mari kita menganalisis algoritma tsb. Pada langkah 2, jika w relatif prima terhadap n maka kita menghitung $w^s, w^{2s}, w^{4s}, \dots$, dengan mengkuadratkan secara berturut-turut, sampai

$$w^{2^t s} \equiv 1 \pmod{n}$$

untuk suatu t . Karena

$$ab - 1 = 2^r s \equiv 0 \pmod{\phi(n)},$$

kita tahu bahwa $w^{2^r s} \equiv 1 \pmod{n}$. Sehingga, loop **while** berakhir setelah paling banyak r kali. Pada akhir loop, kita mendapatkan nilai v_0 yang memenuhi $v_0^2 \equiv 1 \pmod{n}$ tetapi $v_0 \not\equiv 1 \pmod{n}$. Jika $v_0 \equiv -1 \pmod{n}$, maka algoritma gagal; bila tidak, diperoleh v_0 akar non-trivial 1 mod n , sehingga kita dapat memfaktorkan n .

Berikutnya, kita akan menunjukkan bahwa algoritma akan berhasil dengan probabilitas sedikitnya $1/2$. Terdapat dua cara dimana algoritma dapat gagal memfaktorkan n :

- 1 $w^s \equiv 1 \pmod{n}$ (langkah 7)
- 2 $w^{2^t s} \equiv -1 \pmod{n}$ utk suatu t , $0 \leq t \leq r - 1$ (step 11).

Perhatikan $r + 1$ buah kongruensi. Kita akan menghitung banyaknya solusi dari setiap kongruensi. Pertama, perhatikan $w^s \equiv 1 \pmod{n}$. Amati bahwa $x \equiv 1 \pmod{n}$ jika dan hanya jika $x \equiv 1 \pmod{p}$ dan $x \equiv 1 \pmod{q}$. Sehingga, perhatikan $w^s \equiv 1 \pmod{p}$. Karena p prima ganjil, maka $\mathbb{Z}_p^*$ siklis. Misal g elemen primitif mod p . Kita dapat menulis $w = g^u$ untuk suatu u , $0 \leq u \leq p - 2$.

Maka

$$\begin{aligned} w^s &\equiv 1 \pmod{p} \\ g^{us} &\equiv 1 \pmod{p} \\ (p - 1) &\mid us. \end{aligned}$$

Tulis

$$p - 1 = 2^i p_1$$

dimana p_1 ganjil, dan

$$q - 1 = 2^j q_1$$

dimana q_1 ganjil. Karena

$$\phi(n) = (p - 1)(q - 1) \mid (ab - 1) = 2^r s,$$

kita mendapatkan

$$2^{i+j} p_1 q_1 \mid 2^r s.$$

Sehingga

$$i + j \leq r$$

dan

$$p_1 q_1 \mid s.$$

Sekarang, $(p-1) \mid us$ menjadi $2^i p_1 \mid us$. Karena $p_1 \mid s$ dan s ganjil maka $2^i \mid u$. Sehingga, $u = k2^i$, $0 \leq k \leq p_1 - 1$, dan banyaknya solusi dari $w^s \equiv 1 \pmod{p}$ adalah p_1 . Serupa, kita mendapatkan banyaknya solusi dari $w^s \equiv 1 \pmod{q}$ adalah q_1 . Dengan Teorema "Chinese remainder", banyaknya solusi dari $w^s \equiv 1 \pmod{n}$ adalah $p_1 q_1$.

Selanjutnya, pandang $w^{2^t s} \equiv -1 \pmod{n}$ untuk suatu t , $0 \leq t \leq r-1$. Sekali lagi, split menjadi dua kongruensi.

Pertama, pandang $w^{2^t s} \equiv -1 \pmod{p}$. Dengan menulis $w = g$ seperti di atas, kita mendapatkan

$$g^{u2^t s} \equiv -1 \pmod{p}$$

Karena $g^{(p-1)/2} \equiv -1 \pmod{p}$, kita dapat

$$\begin{aligned} u2^t s &\equiv \frac{p-1}{2} \pmod{p-1} \\ (p-1) &\mid \left(u2^t s - \frac{p-1}{2} \right) \\ 2(p-1) &\mid \left(u2^{t+1} s - (p-1) \right). \end{aligned}$$

Karena $p-1 = 2^i p_1$, didapat

$$2^{i+1} p_1 \mid \left(u2^{t+1} s - 2^i p_1 \right).$$

Maka, ini menjadi

$$2^{i+1} \mid \left(\frac{u2^{t+1} s}{p_1} - 2^i \right).$$

Sekarang, jika $t \geq i$ maka tidak terdapat solusi karena $2^{i+1} \mid 2^{t+1}$ tapi $2^{i+1} \nmid 2^i$. Jika $t \leq i - 1$ maka u adalah solusi jika dan hanya jika u kelipatan ganjil dari 2^{i-t-1} (catat bahwa s/p_1 ganjil). Sehingga, banyaknya solusi dalam hal ini adalah

$$\frac{p-1}{2^{i-t-1}} \times \frac{1}{2} = 2^t p_1.$$

Dengan argumentasi serupa, terdapat tepat $2^t q_1$ solusi dari kongruensi $w^{2^t s} \equiv -1 \pmod{q}$. Dengan memakai Teorema "Chinese remainder", kita mendapatkan banyaknya solusi dari $w^{2^t s} \equiv -1 \pmod{n}$ adalah

$$\begin{aligned} &0, \text{ jika } t \geq \min\{i, j\} \\ &2^{2t} p_1 q_1, \text{ jika } t \leq \min\{i, j\} - 1. \end{aligned}$$

Catat bahwa t mempunyai range dari 0 ke $r - 1$. Tanpa mengurangi perumuman misal $i \leq j$; maka banyaknya solusi adalah 0 jika $t \geq i$. Jumlah pilihan "jelek" untuk w paling banyak sama dengan

$$\begin{aligned} p_1 q_1 + p_1 q_1 (1 + 2^2 + 2^4 + \dots + 2^{2(i-1)}) &= p_1 q_1 (1 + (2^{2i} - 1)/3) \\ &= p_1 q_1 \left(\frac{2}{3} + \frac{2^{2i}}{3} \right). \end{aligned}$$

Karena $p - 1 = 2^i p_1$, $q - 1 = 2^j q_1$. Jika $j \geq i \geq 1$ maka $p_1 q_1 < n/4$. Kita juga punya bahwa

$$2^{2i} p_1 q_1 \leq 2^{i+j} p_1 q_1 = (p - 1)(q - 1) < n.$$

Sehingga, diperoleh

$$p_1 q_1 \left(\frac{2}{3} + \frac{2^{2i}}{3} \right) < \frac{n}{6} + \frac{n}{3} = \frac{n}{2}.$$

Karena ada $(n - 1)/2$ pilihan untuk w yg "jelek", ini memberikan bahwa sedikitnya $(n - 1)/2$ pilihan yg "baik" dan sehingga probabilitas sukses dari algoritma tsb adalah sedikitnya $1/2$.

Sistem krypto Rabin

Sistem krypto Rabin adalah *computationally secure* terhadap serangan *chosen-plaintext* dengan syarat bahwa modulus $n = pq$ tidak dapat difaktorkan. Sistem ini dapat dijabarkan sbb.

Misal $n = pq$, $p, q \equiv 3 \pmod{4}$. Misal $\mathcal{P} = \mathcal{C} = \mathbb{Z}_n$, dan definisikan

$$K = \{(n, p, q, B) : 0 \leq B \leq n-1\}.$$

Untuk $K = \{(n, p, q, B)\}$, definisikan

$$e_K(x) = x(x + B) \pmod{n}$$

dan

$$d_K(y) = \sqrt{\frac{B^2}{4} + y} - \frac{B}{2} \pmod{n}.$$

Nilai n dan B **publik** sementara p dan q **rahasia** (*secret*).

Kita akan tunjukkan bahwa fungsi enkripsi e_K tidak satu-satu, sehingga dekripsi tidak dapat dilakukan secara tegas. Jelas, terdapat 4 kemungkinan plaintext yg dapat menjadi hasil dari enkripsi suatu ciphertext yg diberikan. Misal w salah satu dari empat akar kuadrat dari 1 mod n . Misal $x \in \mathbb{Z}_n$. Maka,

$$\begin{aligned}
 e_K(w(x + \frac{B}{2}) - \frac{B}{2}) &= (w(x + \frac{B}{2}) - \frac{B}{2})(w(x + \frac{B}{2}) + \frac{B}{2}) \\
 &= w^2(x + \frac{B}{2})^2 - (\frac{B}{2})^2 \\
 &= x^2 + Bx + \frac{B^2}{4} - \frac{B^2}{4} \\
 &= x^2 + Bx \\
 &= e_K(x)
 \end{aligned}$$

Empat kemungkinan plaintext yg dapat dienkrip menjadi $e_K(x)$ adalah $x, -x - B, w(x + B/2) - B/2$ dan $-w(x + B/2) - B/2$, w akar kuadrat non-trivial dari $1 \pmod n$.

Mari kita lihat pendekripsian dari sisi Bob. Diberikan suatu ciphertext y , Bob ingin menentukan x sehingga

$$x^2 + Bx \equiv y \pmod n.$$

Jika $x = x_1 - B/2$ maka

$$x_1^2 - Bx_1 + \frac{B^2}{4} + Bx_1 - \frac{B^2}{2} - y \equiv 0 \pmod n,$$

atau

$$x_1^2 \equiv \frac{B^2}{4} + y \pmod n.$$

Jika $C = B^2/4 + y$ maka diperoleh

$$x_1^2 \equiv C \pmod{n}.$$

Sehingga, dekripsi tereduksi menjadi proses extracting akar kuadrat mod n . Hal ini ekuivalen dengan memecahkan dua kongruensi

$$x_1^2 \equiv C \pmod{p}.$$

dan

$$x_1^2 \equiv C \pmod{q}.$$

Setiap kongruensi ini menghasilkan dua akar kuadrat dari C . Sehingga, dengan teorema "Chinese remainder" kita punya 4 akar kuadrat dari C . Kita dapat memakai Euler's criterion utk menentukan apakah C residu kuadratik mod p (dan mod q). Tetapi, Euler's criterion tidak dapat membantu kita utk mencari akar kuadrat dari C .

Jika $p \equiv 3 \pmod{4}$, terdapat sebuah formula sederhana utk menghitung akar kuadrat dari residu kuadratik $\pmod{p}$. Misal C residu kuadratik. Maka,

$$\begin{aligned}(\pm C^{(p+1)/4})^2 &= C^{(p+1)/2} \pmod{p} \\ &= C^{(p-1)/2} C \pmod{p} \\ &= C \pmod{p}.\end{aligned}$$

Sehingga, dua akar kuadrat dari $C \pmod{p}$ adalah $\pm C^{(p+1)/4} \pmod{p}$. Serupa, dua akar kuadrat dari $C \pmod{q}$ adalah $\pm C^{(q+1)/4} \pmod{q}$. Maka, dengan theorema "Chinese remainder" kita punya 4 akar kuadrat x_1 dari $C \pmod{n}$.

Sekali kita telah menentukan empat nilai yg mungkin dari x_1 , kita dapat menghitung x dari persamaan $x = x_1 + B/2$ untuk mendapatkan keempat kemungkinan plaintext. Ini memberikan formula dekripsi:

$$d_K(y) = \sqrt{\frac{B^2}{4} + y} - \frac{B}{2}.$$

Contoh 10. Misal $n = 77 = 7 \times 11$ dan $B = 9$. Maka, fungsi enkripsinya adalah

$$e_K(x) = x^2 + 9x \pmod{77}$$

dan formula dekripsi adalah

$$d_K(y) = \sqrt{1 + y} - 43 \pmod{77}.$$

Misal Bob ingin mendekripsi ciphertext $y = 22$. Pertama kali perlu ditentukan akar kuadrat dari $23 \pmod{7}$ dan $\pmod{11}$. Karena 7 dan 11 keduanya kongruen $3 \pmod{4}$, kita punya:

$$23^{(7+1)/4} \equiv 2^2 \equiv 4 \pmod{7}$$

dan

$$23^{(11+1)/4} \equiv 1^3 \equiv 1 \pmod{11}.$$

Dengan teorema "Chinese remainder", kita menghitung 4 akar kuadrat dari $23 \pmod{77}$, yakni $\pm 10, \pm 32 \pmod{77}$. Sehingga, keempat kemungkinan plaintext tsb adalah:

$$10 - 43 \pmod{77} = 44$$

$$67 - 43 \pmod{77} = 24$$

$$32 - 43 \pmod{77} = 66$$

$$45 - 43 \pmod{77} = 2.$$



Sekarang, diskusikan keamanan dari sistem kriptografi Rabin.

Buktikan bahwa sebarang algoritma hipotetikal **A** dapat digunakan sebagai oracle dalam suatu algoritma Las Vegas yg memfaktorkan modulus n dengan probabilitas sedikitnya $1/2$.

Pemfaktoran modulus Rabin, diberikan oracle dekripsi

- ① Pilih bulat random t , $1 \leq t \leq n - 1$
- ② Hitung $y = t^2 - B^2/4 \pmod{n}$
- ③ Call **A**(y), utk mendapatkan dekripsi x
- ④ Hitung $x_1 = x + B/2$
- ⑤ If $x_1 \equiv \pm t \pmod{n}$ then
 quit (failure)
else
 $\gcd(x_1 + t, n) = p$ or q (success)

Periksa bahwa

$$y = e_k(t - \frac{B}{2}),$$

sehingga nilai x akan diberikan pada langkah 3. Catat bahwa dalam langkah 4: $x_1^2 \equiv t^2 \pmod{n}$. Hal ini memberikan $x_1 \equiv \pm t \pmod{n}$, atau $x_1 \equiv wt \pmod{n}$, dimana w salah satu dari akar kuadrat non-trivial dari 1 mod n . Dalam kasus kedua, kita punya

$$n \mid (x_1 - t)(x_1 + t),$$

Tetapi n tidak membagi setiap faktor di sisi kanan. Sehingga, $\gcd(x_1 + t, n)$ atau $\gcd(x_1 - t, n)$ akan memberikan salah satu p atau q . Sehingga, n dapat difaktorkan.

Hitung probabilitas sukses dari algoritma ini. Untuk dua residu tak nol t_1 , t_2 , definisikan

$$t_1 \sim t_2 \iff t_1^2 \equiv t_2^2 \pmod{n}.$$

Sehingga,

- 1 $t \sim t$ untuk semua t
- 2 $t_1 \sim t_2$ menyebabkan $t_2 \sim t_1$
- 3 $t_1 \sim t_2$, $t_2 \sim t_3$ menyebabkan $t_1 \sim t_3$

Hal ini mengatakan bahwa relasi $\sim$ adalah sebuah kelas ekivalensi.

Kelas-kelas ekivalen dari $\mathbb{Z}_n \setminus \{0\}$ semuanya mempunyai kardinalitas 4:

Kelas ekivalen yg memuat t adalah himpunan

$$[t] = \{\pm t, \pm wt \pmod{n}\},$$

dimana w akar kuadrat non-trivial dari 1 mod n .

Setiap dua nilai t dalam kelas ekuivalen yg sama akan memberikan nilai nilai y yg sama. Sekarang, pandang nilai x yg dikembalikan oleh oracle $\mathbf{A}$ utk y yg diberikan. Kita punya:

$$[y] = \{\pm y, \pm wy\}.$$

Jika $t = \pm y$ maka algoritma gagal; Jika $t = \pm wy$ maka algoritma berhasil. Karena t dipilih random, maka probabilitas keberhasilan dari algoritma tsb adalah $1/2$. □

Algoritma-algoritma pemfaktoran

Terdapat tiga algoritma pemfaktoran yg paling efektif untuk bilangan sangat besar, yaitu saringan kuadratik (*quadratic sieve*), algoritma kurva eliptik (*elliptic curve algorithm*), dan saringan lapangan bilangan (*number field sieve*).

Algoritma terkenal lainnya yg merupakan precursor meliputi *Pollard's rho-method*, algoritma $p - 1$, algoritma $p + 1$ Williams, pecahan kontinu (*continued fraction*) dan *trial division*.

Trial division terdiri dari pembagian n dengan setiap bilangan bulat mulai dari 2 hingga $\lfloor \sqrt{n} \rfloor$. Untuk $n < 10^{12}$ metoda faktorisasi ini masih *reasonable*.

Metode $p - 1$

Algoritma $p - 1$ Pollard dikenalkan pada 1974, sbb.

Input: n dan B

- ① $a = 2$
- ② **For** $j = 2$ **to** B **do**
 $a = a^j \pmod n$
- ③ $d = \gcd(a - 1, n)$
- ④ **If** $1 < d < n$ **then**
 d faktor dari n (berhasil)
else
 tidak ada faktor dari n yg ditemukan (gagal)

Misal p pembagi prima dari n , dan $q \leq B$ dan

$$(p - 1) \mid B!.$$

Pada akhir langkah 2,

$$a \equiv 2^{B!} \pmod{n},$$

sehingga, karena $p \mid n$ kita punya

$$a \equiv 2^{B!} \pmod{p},$$

Karena $(p - 1) \mid B!$ dan oleh teorema Fermat didapat:

$$a \equiv 1 \pmod{p}.$$

Maka, dalam langkah 4,

$$p|(a-1)$$

dan $p|n$, sehingga

$$p|d = \gcd(a-1, n).$$

Bilangan bulat d akan merupakan pembagi taktrivial dari n .

Contoh 11. Misal $n = 15770708441$. Jika kita aplikasikan metode $p-1$ dengan $B = 180$, kita menemukan bahwa $a = 11620221425$ dalam langkah 3, dan $d = 135979$. Nyatanya, faktorisasi komplit dari n adalah

$$15770708441 = 135979 \times 115979.$$

Faktorisasi berhasil karena 135978 hanya punya faktor prima "kecil" :

$$135978 = 2 \times 3 \times 131 \times 173.$$

Sehingga, dengan mengambil $B \geq 173$, diperoleh $135978|B!$. □

Dalam algoritma ini, terdapat $B - 1$ perpangkatan modular, dimana setiap perpangkatan membutuhkan paling banyak $2\log_2 B$ perkalian modular dengan kuadrat-dan-kali. Operasi $\gcd$ memerlukan $\mathcal{O}((\log n)^3)$. Sehingga, kompleksitas algoritma tsb adalah $\mathcal{O}(B \log B (\log n)^2 + (\log n)^3)$. Jika bilangan B berukuran $\mathcal{O}((\log n)^i)$ untuk suatu besaran i , maka algoritma bersifat polinomial. Akan tetapi, untuk pilihan B tsb probabilitas sukses akan sangat kecil. Jika kita tingkatkan ukuran dari B sampai $\sqrt{n}$ maka algoritma akan berhasil, akan tetapi hal ini tidak lebih cepat dari *trial division*.

Untuk mendapatkan **RSA** yg tahan terhadap faktorisasi dgn algoritma $p - 1$, kita dapat gunakan $n = pq$, $p = 2p_1 + 1$, $q = 2q_1 + 1$, dengan p_1 , q_1 dua prima besar.

Algoritma Dixon dan Saringan kuadrat

Jika kita dapat menemukan $x \not\equiv \pm y \pmod{n}$ sehingga $x^2 \equiv y^2 \pmod{n}$, maka $\gcd(x - y, n)$ adalah faktor non-trivial dari n . Metode tersebut memanfaatkan **keranjang faktor** (*factor base*), yakni suatu himpunan $\mathcal{B}$ dari bilangan-bilangan prima "kecil". Pertama, kita mencoba utk mencari beberapa bilangan bulat x dimana semua faktor prima dari $x^2 \pmod{n}$ berada dalam keranjang faktor $\mathcal{B}$. Selanjutnya, kalikan semua x tersebut sedemikian sehingga setiap faktor prima dalam $\mathcal{B}$ digunakan sejumlah genap kali. Hal ini memberikan kepada kita suatu kongruensi $x^2 \equiv y^2 \pmod{n}$, yang akan menghasilkan faktorisasi dari n .

Contoh 12. Misal $n = 15770708441$, dan $\mathcal{B} = \{2, 3, 5, 7, 11, 13\}$.
Perhatikan tiga kongruensi:

$$\begin{aligned}8340934156^2 &\equiv 3 \times 7 \pmod{n} \\12044942944^2 &\equiv 2 \times 7 \times 13 \pmod{n} \\2773700011^2 &\equiv 2 \times 3 \times 13 \pmod{n}.\end{aligned}$$

Jika kita kalikan ketiganya, didapat:

$$(8340934156 \times 12044942944 \times 2773700011)^2 \equiv (2 \times 3 \times 7 \times 13)^2 \pmod{n}.$$

Sehingga kita dapat

$$9503435785^2 \equiv 546^2 \pmod{n}.$$

Maka,

$$\gcd(9503435785 - 546, 15770708441) = 115759$$

adalah faktor dari n .



Misal $\mathcal{B} = \{p_1, p_2, \dots, p_B\}$. Misal C sedikit lebih besar dari B (misal, $C = B + 10$). Misal kita punya C kongruensi:

$$x_j^2 \equiv p_1^{\alpha_{1j}} \times p_2^{\alpha_{2j}} \times \dots \times p_B^{\alpha_{Bj}} \pmod{n},$$

untuk $1 \leq j \leq C$. Untuk setiap j , pandang

$$a_j = (\alpha_{1j} \bmod 2, \dots, \alpha_{Bj} \bmod 2) \in (\mathbb{Z}_2)^B.$$

Jika kita dapat mencari subset dari a_j 's yang bila dijumlahkan dalam mod 2 memberikan vektor $(0, \dots, 0)$, maka hasil kali dari x_j 's yang bersesuaian akan menggunakan setiap faktor di $\mathcal{B}$ sebanyak genap kali.

Amati bahwa pencarian subset dari C vektor di atas adalah tidak lebih dari sekedar mencari *kebergantungan linier* (*linear dependence*) atas $\mathbb{Z}_2$ dari vektor-vektor tersebut. Dengan diambil $C > B$, kebergantungan linier tersebut pasti ada, dan dapat dicari dengan eliminasi Gauss. Alasan kenapa kita mengambil $C > B + 1$ adalah karena tidak ada jaminan bahwa setiap kongruensi yg diberikan akan memberikan faktorisasi dari n . Kira - kira 50% dari waktu komputasi kita mendapatkan bahwa $x \equiv \pm y \pmod{n}$. Akan tetapi, bila $C > B + 1$ maka kita dapat mempunyai beberapa kongruensi yg diinginkan. Dengan harapan bahwa sedikitnya satu dari kongruensi tersebut akan menghasilkan faktorisasi.

Sekarang tinggal kita diskusikan bagaimana mendapatkan bilangan bulat x_j sedemikian sehingga $x_j^2 \pmod n$ terfaktorkan secara komplit atas $\mathcal{B}$. Terdapat beberapa metode melakukan hal tsb. Salah satunya adalah yang disebut dengan **saringan kuadratik** (*Quadratic Sieve*) oleh C. Pomerance, yang menggunakan bilangan-bilangan bulat $x_j = j + \lfloor \sqrt{n} \rfloor$, $j = 1, 2, \dots$.

Terdapat *trade-off* di sini: jika $B = |\mathcal{B}|$ besar maka besar kemungkinan bilangan x_j terfaktorkan atas $\mathcal{B}$. Akan tetapi, semakin besar B akan membuat semakin banyak jumlah kongruensi yang harus diperhatikan. Pemilihan B yang optimal adalah kira-kira

$$\sqrt{e^{\sqrt{\ln n \ln \ln n}}},$$

dan ini memberikan waktu running yg diharapkan sebesar

$$\mathcal{O}(e^{(1+o(1))\sqrt{\ln n \ln \ln n}}).$$

dimana $o(1)$ menyatakan suatu fungsi dari n yang mendekati 0 bila $n \rightarrow \infty$.

Saringan lapangan bilangan (*the number field sieve*) (1980's) adalah algoritma pemfaktoran yang lebih baru. Algoritma ini memfaktorkan n dengan cara mengkonstruksi kongruensi $x^2 \equiv y^2 \pmod{n}$ dengan komputasi pada ring bilangan aljabar.

Algoritma Pemfaktoran dalam praktek

Waktu running asimtotik dari saringan kuadratik, kurva eliptik dan saringan lapangan bilangan adalah sbb:

saringan kuadratik	$\mathcal{O}(e^{(1+o(1))\sqrt{\ln n \ln \ln n}})$
kurva eliptik	$\mathcal{O}(e^{(1+o(1))\sqrt{2 \ln p \ln \ln p}})$
saringan lapangan bilangan	$\mathcal{O}(e^{(1.92+o(1))(\ln n)^{1/3}(\ln \ln n)^{2/3}})$

Dalam kasus terjelek, $p \approx \sqrt{n}$ dan waktu running asimtotik dari saringan kuadratik dan kurva eliptik adalah sama. Kurva eliptik lebih berguna bila faktor-faktor prima dari n berbeda ukuran. Brent (1988) berhasil memfaktorkan $2^{2^{11}} - 1$ dengan kurva eliptik.

Untuk memfaktorkan modulus **RSA** ($n = pq$, dimana p, q mempunyai ukuran sama), saringan kuadratik merupakan algoritma yang sangat cocok saat ini. Pada 1983, Davis, Holdridge dan Simmons berhasil memfaktorkan $2^{251} - 1$ dengan saringan kuadratik. Selanjutnya, bilangan 106-digit berhasil difaktorkan oleh Lenstra dan Manasse (1989), dengan komputasi terdistribusi (dikenal juga sebagai "*factoring by electronic mail*"). Akhir - akhir ini, pada April 1994, sebuah bilangan 129-digit yg dikenal oleh **RSA**–129 berhasil difaktorkan oleh Atkins, Graff, Lenstra dan Leyland dengan menggunakan saringan kuadratik. Pemfaktoran ini memerlukan 5000 MIPS-tahun waktu komputasi yang diberikan oleh sekitar 600 peneliti di seluruh dunia.

Beberapa bilangan seperti **RSA**–100, **RSA**–110, . . . , **RSA**–500 termasuk dalam daftar modulus **RSA** yang dipublikasikan pada internet sebagai tantangan dalam algoritma pemfaktoran.

Sistem kriptografi Elgamal didasarkan pada problem **logaritma diskrit** (*discrete logarithm*). Problem logaritma diskrit dalam $\mathbb{Z}$ telah menjadi objek banyak penelitian. Secara umum, problem menjadi sukar bila p dipilih dengan cukup hati-hati. Problem tersebut dapat disajikan berikut ini.

Problem logaritma diskrit dalam $\mathbb{Z}_p$

Misal $I = (p, \alpha, \beta)$, dimana p prima, $\alpha \in \mathbb{Z}_p$ adalah primitif, dan $\beta \in \mathbb{Z}_p^*$.

Tentukan bulat a , $0 \leq a \leq p - 2$ sehingga $\alpha^a \equiv \beta \pmod{p}$.

Kita nyatakan bulat a ini dengan $\log_{\alpha} \beta$.

Sistem kriptografi ElGamal tidak deterministik, karena ciphertext yg dihasilkan bergantung pada plaintext x dan pada nilai random k . Sehingga, akan terdapat beberapa ciphertext yang diperoleh dari enkripsi plaintext yang sama. Sistem ElGamal dapat dilihat pada tabel di bawah.

Sistem kriptografi kunci-publik ElGamal di $\mathbb{Z}_p^*$

Misal p prima sehingga problem log diskrit pada $\mathbb{Z}_p$ *intractable*, dan $\alpha \in \mathbb{Z}_p^*$ elemen primitif.

Misal $\mathcal{P} = \mathbb{Z}_p^*$, $\mathcal{C} = \mathbb{Z}_p^* \times \mathbb{Z}_p^*$, dan definisikan

$$\mathcal{K} = \{(p, \alpha, a, \beta) : \beta \equiv \alpha^a \pmod{p}\}.$$

Nilai p, α dan β : publik dan a rahasia.

Untuk $\mathcal{K} = (p, \alpha, a, \beta)$, dan untuk bilangan random (rahasia) $k \in \mathbb{Z}_{p-1}$, definisikan

$$e_{\mathcal{K}}(x, k) = (y_1, y_2),$$

dimana

$$y_1 = \alpha^k \pmod{p} \text{ dan } y_2 = x\beta^k \pmod{p}.$$

Untuk $y_1, y_2 \in \mathbb{Z}_p^*$, definisikan

$$d_{\mathcal{K}}(y_1, y_2) = y_2(y_1^a)^{-1} \pmod{p}.$$

Contoh 13. Misal $p = 2579$, $\alpha = 2$, $a = 765$, dan sehingga

$$\beta = 2^{765} \pmod{2579} = 949.$$

Misal kita ingin mengirim pesan $x = 1299$. Jika $k = 853$ dipilih secara random, maka

$$\begin{aligned} y_1 &= 2^{853} \pmod{2579}. \\ &= 435, \text{ dan} \end{aligned}$$

$$\begin{aligned} y_2 &= 1299 \times 949^{853} \pmod{2579} \\ &= 2396. \end{aligned}$$

Ketika kita menerima ciphertext $y = (435, 2396)$, kita menghitung

$$\begin{aligned} x &= 2396 \times (435^{765})^{-1} \pmod{2579}. \\ &= 1299. \quad \square \end{aligned}$$

Algoritma untuk problem log diskrit

Kita akan mendiskusikan tiga algoritma untuk problem log diskrit, yaitu the **algoritma Shanks**, **algoritma PohligHellman** dan metode **kalkulus indeks** *index calculus method*.

Jelas, problem **logaritma diskrit** dapat dipecahkan dengan pencarian yg melelahkan *exhaustive search* dalam waktu $\mathcal{O}(p)$ dan memory $\mathcal{O}(1)$ (mengabaikan faktor logaritma). Dengan precomputing semua nilai yg mungkin dari α^a , dan mengurutkan pasangan-pasangan tersebut $(a, \alpha^a \bmod p)$ menurut koordinat keduanya, kita dapat memecahkan problem log diskrit dalam waktu $\mathcal{O}(1)$ dengan precomputation $\mathcal{O}(p)$ dan memori $\mathcal{O}(p)$.

Di bawah ini diberikan algoritma Shanks.

Algoritma Shanks untuk problem log diskrit

- ① Hitung $\alpha^{mj} \pmod{p}$, $0 \leq j \leq m-1$, dan p **prima**
- ② Urutkan m buah pasangan terurut $(j, \alpha^{mj} \pmod{p})$ menurut koordinat keduanya, simpan pada daftar L_1
- ③ Hitung $\beta\alpha^{-i} \pmod{p}$, $0 \leq i \leq m-1$
- ④ Urutkan m buah pasangan terurut $(i, \beta\alpha^{-i} \pmod{p})$ menurut koordinat keduanya, simpan pada daftar L_2
- ⑤ Tentukan pasangan $(j, y) \in L_1$ dan $(i, y) \in L_2$ (yang punya koordinat kedua sama)
- ⑥ Definisikan $\log_{\alpha} \beta = mj + i \pmod{p-1}$.

Misal $m = \lceil \sqrt{p-1} \rceil$. Langkah 1 dan 2 dapat dikomputasi awal (*precomputed*). Sekarang, amati bahwa jika $(j, y) \in L_1$ dan $(i, y) \in L_2$ maka

$$\alpha^{mj} = y = \beta \alpha^{-i},$$

sehingga

$$\alpha^{mj+i} = \beta,$$

seperti yang diinginkan. Sebaliknya, untuk setiap β , kita dapat menulis

$$\log_{\alpha} \beta = mj + i,$$

dimana $0 \leq j, i \leq m-1$. Sehingga, pencarian dalam langkah 5 akan berhasil.

Adalah tidak sukar untuk mengimplementasikan algoritma ini sehingga jalan dalam waktu $\mathcal{O}(m)$ dan dengan memori $\mathcal{O}(m)$ (dengan mengabaikan faktor logaritma).

Algoritma Pohlig-Hellman dapat dijabarkan berikut ini. Misal p prima dan,

$$p - 1 = \prod_{j=1}^k p_j^{c_j},$$

dimana semua p_j prima berbeda. Nilai $a = \log_{\alpha} \beta$ ditentukan (secara tunggal) modulo $p - 1$. Jika kita dapat menghitung $a \bmod p_j^{c_j}$ untuk setiap j , $1 \leq j \leq k$ maka kita dapat menghitung $a \bmod p - 1$ dengan theorema "Chinese remainder". Sehingga, misal q prima,

$$p - 1 \equiv 0 \pmod{q^c}, \text{ dan } p - 1 \not\equiv 0 \pmod{q^{c+1}}.$$

Kita akan menunjukkan bagaimana menghitung nilai

$$x = a \bmod q^c,$$

dimana $0 \leq x \leq q^c - 1$. Kita dapat mengekspresikan x dalam radix q sebagai

$$x = \sum_{i=0}^{c-1} a_i q^i,$$

dimana $0 \leq a_i \leq q - 1$. Lihat bahwa

$$a = x + q^c s$$

untuk suatu bulat s . Langkah pertama dari algoritma adalah menghitung a_0 .

Pertama, kita akan menunjukkan bahwa

$$\beta^{(p-1)/q} \equiv \alpha^{(p-1)a_0/q} \pmod{p}.$$

Untuk melihat ini, catat bahwa

$$\beta^{(p-1)/q} \equiv \alpha^{(p-1)(x+q^c s)/q} \pmod{p},$$

sehingga cukup dibuktikan bahwa

$$\alpha^{(p-1)(x+q^c s)/q} \equiv \alpha^{(p-1)a_0/q} \pmod{p}.$$

Hal ini berlaku jika dan hanya jika

$$\frac{(p-1)(x+q^c s)}{q} \equiv \frac{(p-1)a_0}{q} \pmod{p-1}.$$

Akan tetapi, kita punya

$$\begin{aligned}\frac{(p-1)(x + q^c s)}{q} - \frac{(p-1)a_0}{q} &= \frac{(p-1)}{q}(x + q^c s - a_0) \\ &= \frac{(p-1)}{q}\left(\sum_{i=0}^{c-1} a_i q^i + q^c s - a_0\right) \\ &= \frac{(p-1)}{q}\left(\sum_{i=1}^{c-1} a_i q^i + q^c s\right) \\ &= (p-1)\left(\sum_{i=0}^{c-1} a_i q^{i-1} + q^{c-1} s\right) \\ &= 0 \pmod{p-1},\end{aligned}$$

yang kita ingin buktikan.

Sehingga, kita mulai dengan menghitung $\beta^{(p-1)/q} \bmod p$. Jika

$$\beta^{(p-1)/q} \equiv 1 \pmod{p},$$

maka $a_0 = 0$. Jika tidak, kita berturut-turut menghitung

$$\gamma = \alpha^{(p-1)/q} \pmod{p}, \gamma^2 \pmod{p}, \dots,$$

sampai

$$\gamma^i \equiv \beta^{(p-1)/q} \pmod{p},$$

untuk suatu i . Jika hal ini terjadi, kita punya $a_0 = i$.

Sekarang, jika $c = 1$, kita telah selesai. Jika tidak $c > 1$, proses dilanjutkan dengan menghitung a_1 sbb. Definisikan

$$\beta_1 = \beta \alpha^{-a_0}$$

dan tulis

$$x_1 = \log_{\alpha} \beta_1 \pmod{q^c}.$$

Tidak sukar untuk melihat bahwa

$$x_1 = \sum_{i=1}^{c-1} a_i q^i.$$

Sehingga, hal ini memberikan

$$\beta_1^{(p-1)/q^2} \equiv \alpha^{(p-1)a_1/q} \pmod{p}.$$

Jadi, kita akan menghitung $\beta_1^{(p-1)/q^2} \pmod{p}$, dan kemudian menentukan i yg memenuhi

$$\gamma^i \equiv \beta_1^{(p-1)/q^2} \pmod{p}.$$

Maka, didapat $a_1 = i$.

Jika $c = 2$, kita selesai; bila tidak ulangi proses di atas sebanyak $c - 2$ kali, untuk mendapatkan $a_2, \dots, a_{c-1}$.

Algoritma Pohlig-Hellman untuk menghitung $\log_{\alpha} \beta \pmod{q^c}$ diberikan di bawah ini. Dalam algoritma ini, α primitif $\pmod{p}$, q prima,

$$p - 1 \equiv 0 \pmod{q^c}$$

dan

$$p - 1 \not\equiv 0 \pmod{q^{c+1}}.$$

Algoritma tsb menghitung $a_0, \dots, a_{c-1}$, dimana

$$\log_{\alpha} \beta \pmod{q^c} = \sum_{i=0}^{c-1} a_i q^i.$$

Algoritma Pohlig-Hellman utk menghitung $\log_{\alpha} \beta \pmod{q^c}$.

- ① Hitung $\gamma^i = \alpha^{(p-1)i/q} \pmod{p}$ untuk $0 \leq i \leq q - 1$.
- ② Tetapkan $j = 0$ dan $\beta_j = \beta$.
- ③ **While** $j \leq c - 1$ **do**
- ④ Hitung $\delta = \beta_j^{(p-1)/q^{j+1}} \pmod{p}$
- ⑤ Tentukan i sehingga $\delta = \gamma_i$
- ⑥ $\alpha_j = i$
- ⑦ $\beta_{j+1} = \beta_j \alpha^{-a_j q^j} \pmod{p}$
- ⑧ $j = j + 1$.

Contoh 14. Misal $p = 29$; maka $n = p - 1 = 28 = 2^2 7^1$. Misal $\alpha = 2, \beta = 18$. Hitung $a = \log_2 18$. Kita lanjutkan dengan menghitung $a \bmod 4$ dan kemudian menghitung $a \bmod 7$. Tetapkan $q = 2$ dan $c = 2$. Kemudian,

$$\gamma_0 = 1$$

dan

$$\begin{aligned}\gamma_1 &= \alpha^{28/2} \pmod{29} \\ &= 2^{14} \pmod{29} \\ &= 28;\end{aligned}$$

$$\begin{aligned}\delta &= \beta^{28/2} \pmod{29} \\ &= 18^{14} \pmod{29} \\ &= 28.\end{aligned}$$

Sehingga, $a_0 = 1$. Kemudian, hitung

$$\begin{aligned}\beta_1 &= \beta_0 \alpha^{-1} \pmod{29} \\ &= 9.\end{aligned}$$

$$\begin{aligned}\beta_1^{28/4} \pmod{29} &= 9^7 \pmod{29} \\ &= 28.\end{aligned}$$

Karena $\gamma_1 \equiv 28 \pmod{29}$, didapat $a_1 = 1$. Sehingga, $a \equiv 3 \pmod{4}$.

Selanjutkan, tetapkan $q = 7$, $c = 1$. Kita punya

$$\begin{aligned}\beta^{28/7} \pmod{29} &= 18^4 \pmod{29} \\ &= 25.\end{aligned}$$

$$\begin{aligned}\gamma_1 &= \alpha^{28/7} \pmod{29} \\ &= 2^4 \pmod{29} \\ &= 16.\end{aligned}$$

Selanjutnya, hitung

$$\gamma_2 = 24$$

$$\gamma_3 = 7$$

$$\gamma_4 = 25.$$

Sehingga, $a_0 = 4$, dan $a \equiv 4 \pmod{7}$.

Akhirnya, selesaikan sistem

$$a \equiv 3 \pmod{4}$$

$$a \equiv 4 \pmod{7}$$

dengan teorema "Chinese remainder", didapat $a \equiv 11 \pmod{28}$. Sehingga, diperoleh $\log_2 18$ dalam $\mathbb{Z}_{29}$ sama dengan 11. □

Metode Kalkulus Indeks untuk menghitung log diskrit digunakan dalam beberapa algoritma pemfaktoran yg baik. Metode ini bekerja berdasarkan pada keranjang faktor (**factor base**), yakni suatu himpunan $\mathcal{B}$ terdiri dari prima-prima "kecil". Misal $\mathcal{B} = \{p_1, p_2, \Delta\Delta\Delta, p_B\}$. Langkah-langkah yang dilakukan:

- Tentukan logaritma diskrit dari setiap elemen $p \in \mathcal{B}$.
- Hitung log diskrit dari elemen β yg diinginkan, dengan menggunakan pengetahuan pada langkah pertama.

Dalam *precomputation*, konstruksi $C(= B + 10)$ buah kongruensi modulo p , sbb:

$$\alpha^{x_j} \equiv p_1^{a_{1j}} p_2^{a_{2j}} \cdots p_B^{a_{Bj}} \pmod{p},$$

$1 \leq j \leq C$. Perhatikan bahwa kongruensi-kongruensi tsb dapat ditulis:

$$x_j \equiv a_{1j} \log_{\alpha} p_1 + \cdots + a_{Bj} \log_{\alpha} p_B \pmod{p-1},$$

$1 \leq j \leq C$. Dengan diberikan C buah kongruensi dalam B buah "unknown" $\log_{\alpha} p_i (1 \leq i \leq B)$, kita berharap bahwa terdapat solusi tunggal mod $p-1$. Jika hal ini terjadi, maka kita dapat menghitung logaritma dari setiap elemen di keranjang faktor.

Bagaimana men-generate kongruensi di atas? Satu cara elementer adalah:

- 1 ambil nilai random x ,
- 2 hitung $\alpha^x \pmod{p}$,
- 3 periksa apakah $\alpha^x \pmod{p}$ punya semua faktornya dalam $\mathcal{B}$ (gunakan trial division, misalnya).

Selanjutnya, kita akan menghitung $\log_{\alpha} \beta$ dengan menggunakan algoritma tipe Las Vegas. Pilih bilangan bulat random s ($1 \leq s \leq p-2$) dan hitung

$$\gamma = \beta \alpha^s \pmod{p}.$$

Sekarang, faktorkan γ atas $\mathcal{B}$. Jika hal ini dapat dilakukan, maka didapat:

$$\beta\alpha^s \equiv p_1^{c_1} p_2^{c_2} \cdots p_B^{c_B} \pmod{p}.$$

Atau, diperoleh:

$$\log_\alpha \beta + s \equiv c_1 \log_\alpha p_1 + \cdots + c_B \log_\alpha p_B \pmod{p} - 1).$$

Karena semua telah diketahui sekarang kecuali $\log_\alpha \beta$, maka kita dapat menghitung nilai $\log_\alpha \beta$.

Bit Security of Discrete Logs

Kita akan perhatikan apakah bit individual dari suatu logaritma diskrit mudah atau sukar untuk dihitung. Lebih jelasnya, pandang problem berikut ini.

Bit ke- i dari logaritma diskrit

Misal $I = (p, \alpha, \beta, i)$, p prima, $\alpha \in \mathbb{Z}_p^*$ primitif, $\beta \in \mathbb{Z}_p^*$, dan i bulat, $1 \leq i \leq \lceil \log_2(p-1) \rceil$.

Hitung $L_i(\beta)$, yaitu bit ke- i terkecil dari $\log_\alpha \beta$.

Pertama, kita akan tunjukkan bahwa menghitung $L_1(\beta)$ mudah.
Perhatikan pemetaan $f : \mathbb{Z}_p^* \rightarrow \mathbb{Z}_p^*$ yang didefinisikan sbb:

$$f(x) = x^2 \pmod{p}.$$

Misal $QR(p)$ himpunan residu kuadratik mod p ; maka

$$QR(p) = \{x^2 \pmod{p} : x \in \mathbb{Z}_p^*\}.$$

Periksa bahwa $f(x) = f(p - x)$. Selanjutnya, catat

$$w^2 \equiv x^2 \pmod{p} \iff p \mid (w - x)(w + x) \pmod{p}.$$

Hal ini terjadi jika dan hanya jika

$$w \equiv \pm x \pmod{p}.$$

Sehingga, memberikan

$$|f^{-1}(y)| = 2, \text{ for every } y \in QR(p).$$

Jadi,

$$|QR(p)| = \frac{p-1}{2}.$$

Ini berarti bahwa tepat separuh dari residu dalam $\mathbb{Z}_p^*$ adalah residu kuadratik dan separuh lainnya tidak.

Misal α primitif dalam $\mathbb{Z}_p$. Maka, $\alpha^a \in QR(p)$ bila a genap. Karena $(p-1)/2$ buah elemen $\alpha^0 \pmod{p}, \alpha^2 \pmod{p}, \dots, \alpha^{p-3} \pmod{p}$ semuanya berbeda, maka:

$$QR(p) = \{\alpha^{2i} \pmod{p} : 0 \leq i \leq (p-3)/2\}.$$

Sehingga β residu kuadratik jika dan hanya jika $\log_{\alpha} \beta$ genap, yakni jika dan hanya jika $L_1(\beta) = 0$. Ingat bahwa, dengan Euler's criterion, kita punya β residu kuadratik jika dan hanya jika

$$\beta^{(p-1)/2} \equiv 1 \pmod{p}.$$

Sehingga, diperoleh $L_1(\beta) = \begin{cases} 0 & \beta^{(p-1)/2} \equiv 1 \pmod{p} \\ 1 & \text{lainnya} \end{cases}$



Sekarang, perhatikan komputasi $L_i(\beta)$, $i \geq 2$. Misal

$$p - 1 = 2^s t, \text{ dimana } t \text{ ganjil.}$$

Maka, dapat ditunjukkan bahwa menghitung $L_i(\beta)$ mudah jika $i \leq s$. Di sisi lain, menghitung $L_{s+1}(\beta)$ (kemungkinan) sukar, dalam arti bahwa sebarang algoritma hipotetikal (atau oracle) utk menghitung $L_{s+1}(\beta)$ dapat digunakan untuk menghitung log diskrit dalam $\mathbb{Z}_p$.

Kita akan tunjukkan utk kasus $s = 1$. Lebih tepatnya, jika $p \equiv 3 \pmod{4}$ prima maka kita tunjukkan bahwa sebarang oracle untuk menghitung $L_2(\beta)$ dapat dipakai utk memecahkan problem **log diskrit** dalam $\mathbb{Z}_p$.

Jika β residu kuadratik pada $\mathbb{Z}_p$ dan $p \equiv 3 \pmod{4}$ maka dua akar kuadrat dari $\beta \pmod{p}$ adalah $\pm\beta^{(p+1)/4} \pmod{p}$. Penting utk dicatat bahwa, untuk sebarang $\beta \neq 0$,

$$L_1(\beta) \neq L_1(p - \beta), \text{ jika } p \equiv 3 \pmod{4}.$$

Kita akan membuktikan hal itu di bawah ini. Misal

$$\alpha^a \equiv \beta \pmod{p};$$

maka

$$\alpha^{a+(p-1)/2} \equiv -\beta \pmod{p}.$$

Karena $p \equiv 3 \pmod{4}$, maka $(p-1)/2$ ganjil, dan klaim terpenuhi.

Sekarang, misal $\beta = \alpha^a$ untuk suatu pangkat genap a (tak diketahui). Maka, salah satu dibawah ini berlaku:

$$\beta^{(p+1)/4} \equiv \alpha^{a/2} \pmod{p},$$

atau

$$-\beta^{(p+1)/4} \equiv \alpha^{a/2} \pmod{p}.$$

Kita dapat menentukan salah satu dari dua di atas yg benar jika kita tahu nilai $L_2(\beta)$, karena

$$L_2(\beta) = L_1(\alpha^{a/2}).$$

Menghitung log diskrit pada $\mathbb{Z}_p$ utk $p \equiv 3 \pmod{4}$,
diberikan oracle utk $L_2(\beta)$

- 1 $x_0 = L_1(\beta)$
- 2 $\beta = \beta/\alpha^{x_0} \pmod{p}$
- 3 $i = 1$
- 4 **While** $\beta \neq 1$ **do**
- 5 $x_1 = L_2(\beta)$
- 6 $\gamma = \beta^{(p+1)/4} \pmod{p}$
- 7 **If** $L_1(\gamma) = x_i$ **then**
- 8 $\beta = \gamma$
- 9 **else**
- 10 $\beta = p - \gamma$
- 11 $\beta = \beta/\alpha^{x_i} \pmod{p}$
- 12 $i = i + 1$

Pada akhir algoritma, x_i memberikan bit-bit dalam representasi biner dari $\log_{\alpha} \beta$, yakni

$$\log_{\alpha} \beta = \sum_{i \geq 0} x_i 2^i.$$

Tulis

$$x = \log_{\alpha} \beta = \sum_{i \geq 0} x_i 2^i.$$

Untuk $i \geq 0$, definisikan

$$Y_i = \lfloor \frac{x}{2^{i+1}} \rfloor.$$

Definisikan β_0 sebagai nilai dari β pada langkah 2, dan β_i , $i \geq 1$, sebagai nilai β pada langkah 11. Dapat ditunjukkan bahwa

$$\beta_i = \alpha^{2^{Y_i}} \pmod{p}, \text{ for all } i \geq 0.$$

Karena

$$2Y_i = Y_{i-1} - x_i,$$

didapat

$$x_{i+1} = L_2(\beta_i), \quad i \geq 0.$$

Karena $x_0 = L_1(\beta)$ maka algoritma terbukti benar. □